Lab experiment-1

8-Puzzle Problem

Aim

To solve the 8-Puzzle problem using the Breadth-First Search (BFS) algorithm.

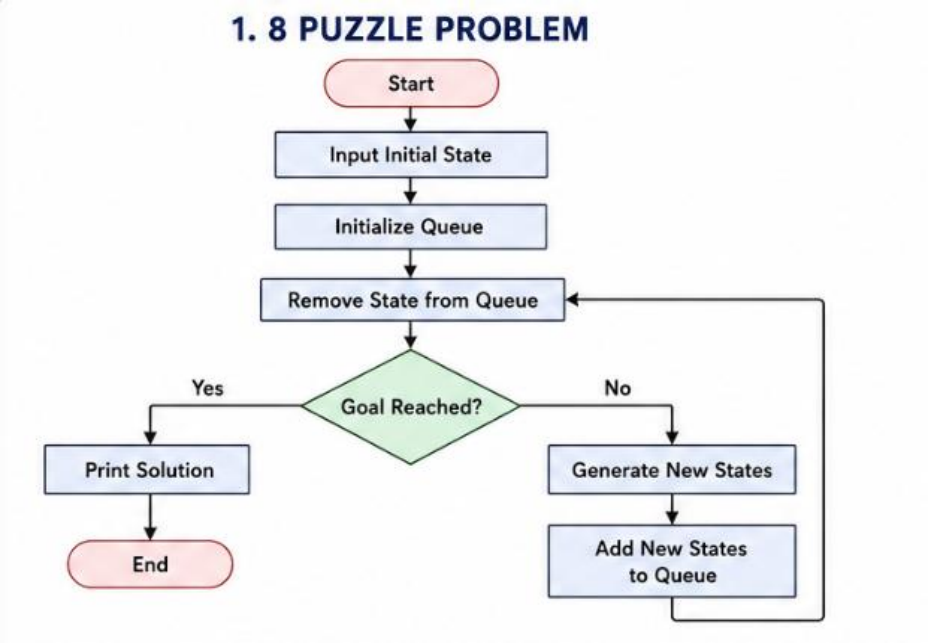
Objective

To find the shortest path from the initial state to the goal state by moving the blank tile.

Algorithm

1. Start with the initial puzzle state.
2. Insert the state into a queue.
3. Remove the first state from the queue.
4. Check whether it is the goal state.
5. If not, generate all possible next states.
6. Add unvisited states to the queue.
7. Repeat until the goal state is reached.

Flowchart



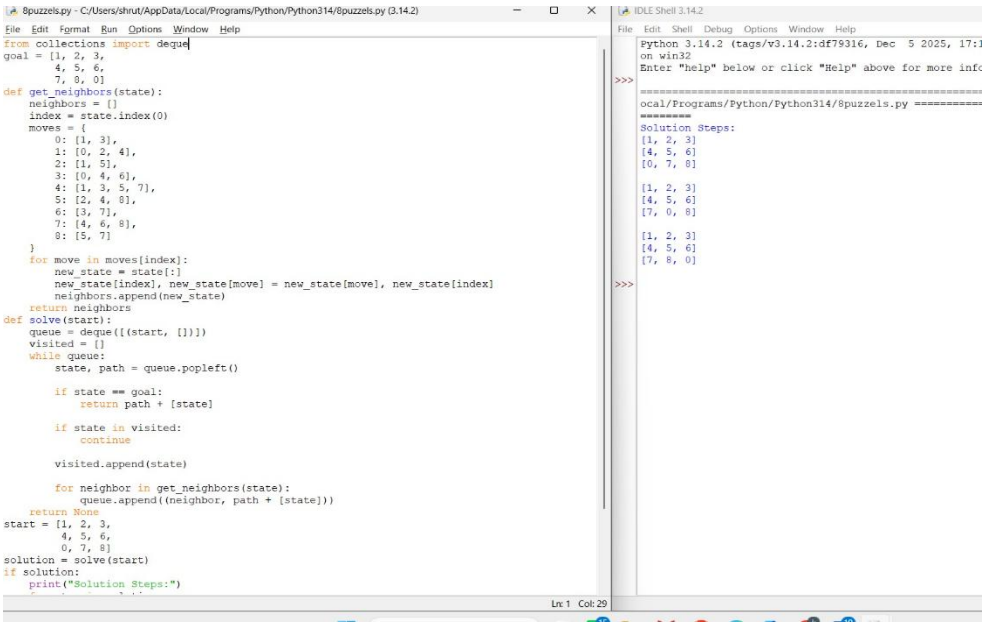
Code

```
from collections import deque
goal = [1, 2, 3,
        4, 5, 6,
        7, 8, 0]
def get_neighbors(state):
    neighbors = []
    index = state.index(0)


    moves = {
        0: [1, 3],
        1: [0, 2, 4],
        2: [1, 5],
        3: [0, 4, 6],
        4: [1, 3, 5, 7],
```

```
        5: [2, 4, 8],
        6: [3, 7],
        7: [4, 6, 8],
        8: [5, 7]
    }
    for move in moves[index]:
        new_state = state[:]
        new_state[index], new_state[move] = new_state[move], new_state[index]
        neighbors.append(new_state)

    return neighbors
def solve(start):
    queue = deque([(start, [])])
    visited = []
    while queue:
        state, path = queue.popleft()
        if state == goal:
            return path + [state]
        if state in visited:
            continue
        visited.append(state)
        for neighbor in get_neighbors(state):
            queue.append((neighbor, path + [state]))
    return None
start = [1, 2, 3,   4, 5, 6,
         0, 7, 8]
```

```
solution = solve(start)
if solution:
    print("Solution Steps:")
    for step in solution:
        print(step[0:3])
        print(step[3:6])
        print(step[6:9])
        print()
else:
    print("No Solution Found")
```

Output



Result

The 8-Puzzle problem is solved successfully using BFS.

Conclusion

The BFS algorithm finds the shortest path to reach the goal state.